If you want to be able to compare two objects of the same class in general, all you need to do is implement the <=> operator and include the Comparable module, like so:

built_in_data/team.rb

```
class Team
  include Comparable
  attr_accessor :wins, :losses, :name

  def initialize(name, wins, losses)
    @name = name
    @wins = wins
    @losses = losses
  end

  def percentage = (wins * 1.0) / (wins + losses)

  def <=>(other)
    raise ArgumentError unless other.is_a?(Team)
    percentage <=> other.percentage
  end

  def to_s = name
end

brewers = Team.new("Brewers", 73, 89)
cardinals = Team.new("Cardinals", 86, 76)
cubs = Team.new("Cubs", 103, 58)
pirates = Team.new("Pirates", 78, 83)
reds = Team.new("Reds", 68, 94)

puts cubs > cardinals
puts "\n"
puts cardinals.between?(cubs, reds)
puts "\n"
puts [brewers, cardinals, cubs, pirates, reds].sort
```

produces:

```
true

false

Reds
Brewers
Pirates
Cardinals
Cubs
```

In this case, we're using the <=> to order teams based on their winning percentage, so using sort on an array of teams gives you the standings, lowest to highest.

The comparable module defines <, <=, >, >=, and == as methods that can be used as operators. It also defines between? as a boolean, and a method called clamp. The call obj.clamp(low, high) takes two values or a range and returns the object if it is between the low and high values, but returns the border value if the clamp is not between the two values:

```
7.clamp(5, 10)  # => 7
3.clamp(5, 10)  # => 5
12.clamp(5, 10) # => 10
7.clamp(5..10)  # => 7
```